

Firestore Migration Plan

Firestore is used across web, mobile and backend for login, live data, file storage and analytics. Some of it is worth keeping; some adds cost and complexity we no longer need. This is what we want to change and why. How to implement it is the dev's call.

10 July 2026

Goal

Move live data and file storage off Firestore and onto our own backend and DigitalOcean Spaces, so everything runs through one system with one source of truth. Keep Firestore only for login. No new vendor, no extra hosting cost — the Spaces bucket already exists.

What to keep and what to remove

SERVICE	DECISION	WHY
Auth (login)	KEEP	Handles all login flows on web and mobile — email, Google, Microsoft, magic link, password reset. Reliable, and replacing it is high-risk for no real gain.
Analytics	KEEP	Useful and lightweight. Keep it, but load it only after cookie consent — see the note below.
Live sessions & notifications	REMOVE	Currently synced through a separate Firestore database. We already have all this data in our own database. Replace with the app polling our API every few seconds. We do not need true real-time here.
File storage	REMOVE	Avatars, session recordings, logos and firmware are stored in Firestore. Move them to our DigitalOcean Spaces so all files live in one place we control.
App abuse protection	REMOVE	Only useful while we depend on Firestore's live database and storage. Once those are gone, it protects nothing. Replace with proper API security on our backend.

Order of work

1

Notifications LOW RISK

Switch notifications to polling our own API instead of Firestore. Most of the backend already exists; it needs a small extension so the app can also read already-seen notifications and an unread count. Good first step — low risk, and it proves the polling approach.

2**Live session status** **MEDIUM RISK**

Have the mobile app send live session status to our backend, and have the web read it by polling — instead of both going through Firebase. Once that works, remove the Firebase pieces on all three sides. Note: the mobile app is the one sending this data today, so this touches mobile as well as web and backend.

3**Lock down file storage first** **BEFORE STEP 4**

Our storage bucket currently makes files public by default. Before moving any private session recordings there, make them private and served through secure, time-limited links. This must happen before step 4, not after.

4**Move file uploads to our storage** **BIGGEST PIECE**

Route all uploads — avatars, session recordings, logos, firmware — through our backend into DigitalOcean Spaces. This is the largest part of the work and touches web, mobile and backend, so it should be estimated on its own before we commit. We also need to decide whether to move old existing files across or keep the old ones where they are.

5**Remove leftover Firebase pieces** **EASY FINISH**

Once live data and storage are off Firebase, remove the abuse-protection service and drop the unused Firebase code from the apps. The apps get smaller and faster to load. Login stays on Firebase.

Two things to handle separately

PRIVACY**Analytics runs before cookie consent**

Analytics currently starts the moment the page loads, before the user accepts anything. For an EU-facing product this is a compliance gap. It should be gated behind a cookie-consent step. Separate task, worth doing regardless of this migration.

SECURITY**API needs proper access control**

Our API currently allows access by default unless a developer explicitly locks each endpoint down. Before we remove Firebase's abuse protection, we need real login-required-by-default rules and basic rate limiting on the backend. Separate task, but it should land before or alongside step 5.

What we're not doing

Not replacing login — Firebase Auth stays, no disruption to sign-in. Not rebuilding true real-time — polling every few seconds is enough for session monitoring. Not switching cloud providers — storage moves to the DigitalOcean Spaces we already pay for.